

Pulse Health Platform

Plano de Escalabilidade — 6º Pipeline: Compras

Documento de suporte para implementação futura. Cobre modelagem de negócio, arquitetura de dados, dependência cross-sistema, passo a passo de implementação, riscos conhecidos e checklist de execução.

Autor: Bruno Queles

Repositório: poc-pulse-health-observability

Status: planejamento — não implementado. Preparado para execução futura.

1. Objetivo e contexto

Este documento planeja a adição de um 6º pipeline à Pulse Health Platform: **Compras** (Procurement), representando a área que adquire matéria-prima para a Manufacturing (ERP). O objetivo é duplo: (1) demonstrar que o desenho config-driven do projeto continua escalando sem reescrever arquitetura — mesma prova já feita com o 5º pipeline (Distribution, ADR-017); (2) introduzir, pela primeira vez no projeto, uma **dependência de negócio real anterior à produção** — hoje Manufacturing produz sem verificar disponibilidade de matéria-prima; com Compras, essa lacuna de realismo é fechada.

Este documento é só planejamento. Nenhum código foi escrito ainda. A intenção é ter o desenho completo pronto para execução quando houver tempo disponível, seguindo exatamente a mesma disciplina (ADR, schema, spike isolado, teste em 3 camadas, documentação) já aplicada em todas as expansões anteriores do projeto.

2. O que já seria reaproveitado sem alteração

A maior parte da infraestrutura do projeto é genérica por desenho — não sabe nem precisa saber quantos sistemas existem. A tabela abaixo resume o que funcionaria imediatamente, sem tocar em uma linha de código.

Peça	Reaproveita como está?	Por quê
SimuladorDeSistema (classe base)	Sim, 100%	SimuladorCompras herda dela, igual aos outros 5
sujeira_intencional.py	Sim, 100%	Funções puras, agnósticas a qual sistema chama
limpeza_utils.py	Sim, 100%	Idem
IngestorAutoloader	Sim, 100%	Parametrizado por sistema/tabela, nunca hardcoded
transformar_bronze_para_silver	Sim, 100%	Guiado por configuração declarativa
promover_seed	Sim, 100%	Idem, para seeds
registrar_execucao_pipeline	Sim, 100%	Já genérico por pipeline/item
NotificadorBase/Tabela/Email	Sim, 100%	Não têm noção de "sistema" nenhuma
Asset Bundles (Jobs)	Sim, sem alteração de YAML	Orquestradores já fazem loop por sistema; um sistema a mais é absorvido automaticamente

3. Modelagem de negócio — Compras

Compras representa o processo de aquisição de matéria-prima junto a fornecedores externos, necessária antes de qualquer lote de produção (Manufacturing) poder ser fabricado.

3.1 Entidades e tabelas propostas

Tabela	Tipo	Descrição
compras_fornecedores	Seed (dimensão fixa)	Catálogo de fornecedores — nome, região, prazo médio de entrega, produto/matéria-prima que fornece
compras_pedidos_compra	Evento diário	Pedido de matéria-prima feito a um fornecedor — quantidade, data do pedido, prazo previsto
compras_recebimento	Evento diário	Confirmação de entrada de matéria-prima no estoque — quantidade recebida, data, referência ao pedido de compra

3.2 Calendário de operação

Proposta: Segunda a Sexta (mesmo calendário de Manufacturing) — decisão coerente com o propósito do sistema: não faz sentido comprar matéria-prima em dias que a fábrica não opera. **dias_operacionais = {0, 1, 2, 3, 4}**.

3.3 Volumetria proposta (premissa, a validar como as demais áreas)

Tabela	Volumetria sugerida
compras_fornecedores (seed)	5 fornecedores fixos, 1-2 por produto do catálogo (PROD-001/002/003)
compras_pedidos_compra	~10-25 pedidos/dia
compras_recebimento	Depende do prazo de entrega do fornecedor (nem todo pedido tem recebimento no mesmo dia — ver seção 3.5)

3.4 Sujeira intencional proposta (reaproveitando sujeira_intencional.py)

Campo	Sujeira	Função reaproveitada
data_pedido / data_recebimento	Formato duplo (DD/MM/AAAA vs. AAAA-MM-DD)	formatar_data_suja
quantidade_solicitada / quantidade_recebida	Padrão brasileiro com vírgula	formatar_numero_sujo
status_pedido, fornecedor_nome	Caixa/espaco inconsistente	formatar_texto_sujo
data_recebimento (quando ainda não chegou)	Nulo variado	valor_nulo_variado

3.5 Regra de negócio nova — a mais importante desta expansão

Diferença estrutural em relação a todas as expansões anteriores: até hoje, nenhum sistema do projeto tinha uma restrição real de disponibilidade condicionando outro. Com Compras, propõe-se que **Manufacturing (ERP) só produza um lote se houver matéria-prima suficiente recebida** — isso significa que `erp_lotes_producao` passaria a fazer um cross-read em `compras_recebimento` (mesmo dia), verificando saldo disponível antes de decidir a quantidade produzida, e não apenas gerar um número aleatório dentro de um intervalo fixo como hoje.

Essa regra introduz a primeira **restrição real de negócio entre sistemas** do projeto — até então, toda dependência cross-sistema era sobre referência de chave (ex.: `pedido_id`, `lote_id`), nunca sobre uma condição que altera o comportamento de geração (quanto produzir). Vale destacar isso explicitamente numa eventual apresentação técnica — é um salto de sofisticação de modelagem, não só mais um sistema no mesmo padrão dos anteriores.

4. Nova cadeia de dependência (extensão do ADR-011)

A ordem de execução (`SimuladorFactory.ordem_execucao()`) precisaria ganhar Compras como primeiro elo, antes do ERP — já que Manufacturing passaria a depender de matéria-prima disponível:

Ordem atual (5 pipelines):

CRM -> ERP -> Distribution -> TMS -> Financeiro

Ordem proposta (6 pipelines):

CRM -> Compras -> ERP -> Distribution -> TMS -> Financeiro

Compras não depende de nenhum outro sistema (não precisa ler CRM nem nenhum outro) — pode ser gerado de forma independente, no início da cadeia, junto com CRM (ou logo após). ERP (Manufacturing) passa a depender de Compras, além de continuar sem dependência de CRM (Manufacturing nunca dependeu de pedido, só Distribution/notas de expedição dependem).

5. Impacto em Bronze, Silver e Gold

3 tabelas novas de evento (`compras_pedidos_compra`, `compras_recebimento` — mais o seed `compras_fornecedores`) entram no mesmo padrão config-driven já existente:

Camada	O que muda
Bronze	Nova entrada em TABELAS_POR_SISTEMA ("compras": [...]) nos orquestradores <code>ingerir_dados/gerar_dados/limpar_landing_zone</code>
Silver	Novas entradas em CONFIGURACAO_TABELAS (<code>colunas_data</code> , <code>colunas_numero</code> , <code>colunas_texto</code> , <code>colunas_fk_nulavel</code>) e em CONFIGURACAO_SEEDS
Gold (novo indicador sugerido)	<code>gold_disponibilidade_materia_prima</code> ou <code>observability_producao_limitada_por_estoque</code> — quantos dias/lotos tiveram produção reduzida por falta de matéria-prima

6. Passo a passo de implementação sugerido

Sequência em fases pequenas e testáveis, seguindo a mesma disciplina usada em todas as expansões anteriores do projeto (validar isolado antes de integrar, nunca tocar produção viva sem teste prévio).

Fase 0 — Documentação prévia

- Adicionar Área 6 (Compras) em business-context.md: processo-chave, volumetria, calendário
- Criar docs/schemas/compras.md (mesmo padrão de distribution.md)
- Criar adr-019-sexto-pipeline-compras.md, cobrindo as decisões desta seção

Fase 1 — SimuladorCompras isolado

- Criar src/simuladores/simulador_compras.py (gerar_seed: fornecedores; gerar_registros: pedidos + recebimento)
- Testar isolado em spike novo (src/spikes/teste_simulador_compras), sem tocar em SimuladorERP ainda
- Confirmar geração e gravação real na Landing Zone (mesmo teste de 3 camadas já usado no projeto)

Fase 2 — Integração no Factory (sem tocar no ERP ainda)

- Registrar "compras" em SimuladorFactory._REGISTRO
- Atualizar SimuladorFactory.ordem_execucao() para a nova cadeia (seção 4)
- Testar SimuladorFactory.criar("compras", ...) isoladamente

Fase 3 — Nova regra de negócio no ERP (a parte de maior risco)

- Modificar _gerar_lotes_producao para ler compras_recebimento do dia via cross-read (mesmo padrão do ADR-011)
- Implementar a lógica de limitação de quantidade_produzida por saldo disponível
- Testar em spike separado, comparando lotes gerados COM e SEM matéria-prima suficiente propositalmente, para confirmar que a regra dispara nos dois sentidos

Fase 4 — Atualizar os 6 orquestradores

- gerar_dados, ingerir_dados, limpar_landing_zone: adicionar "compras" às listas de sistema e ao dropdown (lembrar do passo de remover/recriar Widget, já documentado como necessário)
- promover_seeds: adicionar compras_fornecedores em CONFIGURACAO_SEEDS
- construir_gold: adicionar o novo indicador de disponibilidade de matéria-prima

Fase 5 — Ingestão, transformação e testes

- Adicionar entradas em CONFIGURACAO_TABELAS (Silver)
- Testar IngestorAutoloader e transformar_bronze_para_silver isoladamente para as 3 tabelas novas
- Escrever testes automatizados novos (tests/test_simulador_compras.py — mesmo padrão de propriedade usado para sujeira_intencional)

Fase 6 — Reset e backfill (nunca em cima de produção viva)

- Pausar job_diario (pause_status: PAUSED) ANTES de qualquer mudança estrutural entrar na cadeia real
- Reset completo (mesmo procedimento do backfill de 5 pipelines) — não tentar "encaixar" Compras em cima do histórico de 5 pipelines já existente
- Backfill novo, agora com 6 pipelines, com registrar_execucao_pipeline já embutido desde o início (nunca retroativo — Lição 15)
- Validação matemática por calendário, mesmo método já usado 2 vezes no projeto

Fase 7 — Reativação e documentação final

- Testar job_diario completo via bundle run antes de reativar o agendamento
- Despausar (UNPAUSED) só depois de confirmado
- Atualizar architecture.md, README.md, e revisar o AI/BI Dashboard com o novo indicador

7. Riscos conhecidos (baseados em lições já documentadas no projeto)

Risco	Lição relacionada	Mitigação já conhecida
Checkpoint do Autoloader preso a caminho antigo se alguma tabela mudar de pasta	Lição 16 (LOCATION_OVERLAP)	Não aplicável a Compras (tabelas nascem na pasta certa desde o início) — só relevante se algo for reorganizado depois
Dado gerado "por fora" do orquestrador oficial não registra em pipeline_runs	Lição 15	Sempre gerar via SimuladorFactory/orquestrador oficial, nunca via loop manual solto, mesmo em teste
sys.path quebra em mode: production	Lição 18	Reaproveitar a célula de detecção dinâmica já validada nos 6 notebooks existentes — não reinventar
Widget de dropdown não atualiza sozinho ao adicionar novo sistema	Descoberto na Fase 4 do 5º pipeline	dbutils.widgets.remove() seguido de recriação, já documentado como passo necessário
Testar mudança estrutural em cima de produção ativa	ADR-010	Sempre pausar o Job, nunca desenvolver/testar com agendamento automático ligado
Regra de negócio nova (limitação por matéria-prima) pode gerar volumetria muito diferente do esperado	Nenhuma anterior — risco novo desta expansão	Testar isoladamente em spike antes de integrar, comparando distribuição de quantidade_produzida antes/depois da regra

8. Estimativa de esforço (comparável ao 5º pipeline)

Frente	Esforço relativo	Observação
Código (simulador, configs)	Baixo-médio	Reaproveita quase tudo — mesmo padrão do Distribution
Modelagem de negócio	Médio-alto	Definir volumetria, prazos de entrega, regra de limitação exige mais decisão do que código
Nova regra de negócio no ERP	Médio	Único ponto genuinamente novo de lógica, não reaproveitável de nenhuma expansão anterior
Testes automatizados	Baixo	Mesmo padrão de propriedade já usado, 3 arquivos existentes servem de modelo direto
Documentação	Médio	1 ADR novo, 1 schema novo, atualização de business-context, architecture, README

9. Checklist resumido de execução

- ☐ business-context.md — Área 6 (Compras) documentada
- ☐ docs/schemas/compras.md criado
- ☐ adr-019-sexto-pipeline-compras.md criado (Status: Proposto, depois Aceito)
- ☐ simulador_compras.py criado e testado isolado (spike, 3 camadas)
- ☐ SimuladorFactory atualizado (registro + nova ordem de dependência)
- ☐ Regra de limitação de matéria-prima implementada em simulador_erp.py e testada isoladamente
- ☐ 6 orquestradores atualizados (listas de sistema, Widgets recriados, CONFIGURACAO_TABELAS/SEEDS)
- ☐ Novo indicador Gold/observability implementado
- ☐ Testes automatizados novos escritos e passando
- ☐ job_diario PAUSADO antes de qualquer mudança estrutural entrar na cadeia real
- ☐ Reset completo executado (não tentar encaixar em cima do histórico existente)
- ☐ Backfill novo com 6 pipelines, pipeline_runs registrado desde o início
- ☐ Validação matemática por calendário confirmada
- ☐ job_diario testado via bundle run antes de reativar
- ☐ job_diario despausado (UNPAUSED) só após confirmação
- ☐ architecture.md, README.md e AI/BI Dashboard atualizados

Documento de planejamento — Pulse Health Platform. Nenhuma parte deste plano foi implementada no momento da criação deste documento. Preparado para execução futura, seguindo a mesma disciplina de engenharia (ADR, spike, teste em camadas, documentação) já aplicada em todas as expansões anteriores do projeto.